Doutiva — Plano de implementação

Cérebro c
7 de nove

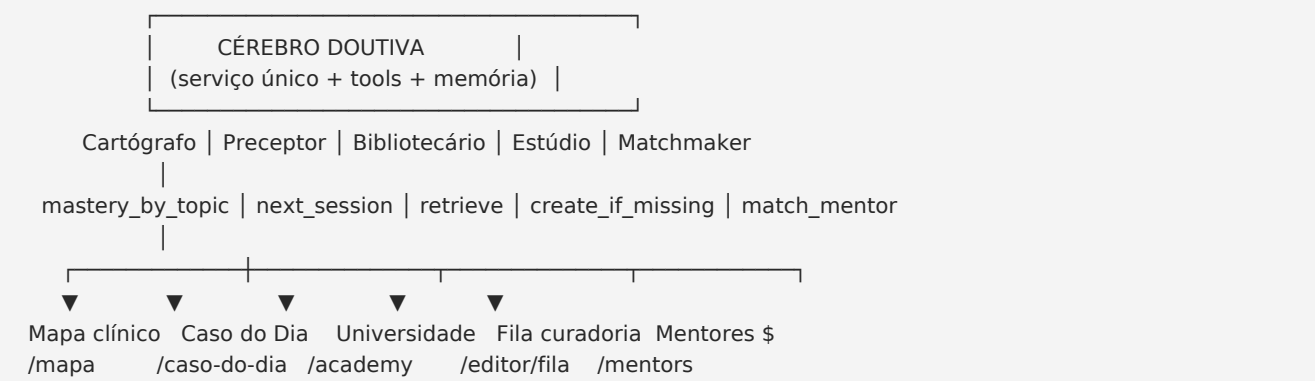
0. O que já existe (não reinventar)

Peça | Estado hoje | Onde

- Tutor IA (aula) · Funciona (RAG + citações) · tutor.server.ts, /api/tutor
- Professores IA · Funciona (4 personas + mastery_records) · preceptor.server.ts, /professores
- Caso do Dia · Funciona, mas calendário global · daily-case.ts, /caso-do-dia
- Revisão espaçada · Funciona (SM-2 simples) · review.ts, /revisao
- Estúdio (admin) · Funciona (gera rascunho, exige reviewed_by) · lesson-ai.server.ts, /editor

… +8 linhas no markdown completo

1. Arquitetura-alvo (um cérebro)



Invariantes (não negociar)

- 1. Um médico = um mapa persistido (mastery_by_topic).
- 2. Conteúdo clínico permanente exige assinatura humana (nome + CRM).
- 3. IA não entrega conduta antes do aluno se comprometer (socrático).
- 4. Aluno ≠ admin ≠ mentor (papéis claros na UI e no servidor).
- 5. Mentoria paga passa pela plataforma (take rate); sem bypass.

2. Ondas de entrega (o “tudo” em ordem de dependência)

Onda | Nome | Resultado para o médico | Portão para avançar

- 0 · Fundação · IA no ar, aluno QA, dados limpos · Health OK + login aluno
- 1 · Memória + Cartógrafo · Tem mapa verde/amarelo/vermelho · ≥80% do piloto completa mapa
- 2 · Preceptor + Caso do Dia 2.0 · Caso diário vem do seu mapa · ≥70% fazem 3 sessões/sem
- 3 · Universidade adaptativa · Curso = fila do que falta · Domínio médio +15 pts
- 4 · Bibliotecário + Estúdio vivo · Gap → busca → cria → curadoria → reuso · % reuso > criação após 4 sem

… +2 linhas no markdown completo

Cada onda tem: schema → serviço → UI → QA → métrica. Não abrir a próxima onda se o portão falhar — corrige, não empilha.

Onda 0 — Fundação (pré-requisito)

0.1 Ambiente

- OPENAI_API_KEY (+ Anthropic se Estúdio) na Vercel
- SUPABASE_URL / SUPABASE_ANON_KEY / SUPABASE_SERVICE_ROLE_KEY
- Redeploy; GET /api/health confirma chaves
- Conta aluno (sem admin) para QA
- Remover/ocultar fixtures de teste óbvias

0.2 Docente real

- Pelo menos 1 médico com nome + CRM no “Revisado por” das aulas canônicas

Pronto quando: Tutor e Professores respondem em produção; aluno QA vê campus limpo.

Onda 1 — Memória + Cartógrafo (Mapa clínico)

Nome: Cartógrafo

O que é: entrevista que vira mapa de domínio do médico.

Como será feito:

Schema

```
mastery_by_topic
  user_id, topic_id, score 0–100, band (green|yellow|red),
  confidence_avg, attempts, last_seen_at, updated_at

topics
  id, slug, title, specialty, parent_id?, course_ids[]

topic_edges (opcional nesta onda, obrigatório na 3)
  from_topic, to_topic, relation (prereq|related)
```

Serviço (`src/lib/brain/`)

- brain.server.ts — router do cérebro
- Tools: getMastery, updateMastery, listTopics
- cartographer.server.ts — prompt socrático + parsing → updates em mastery_by_topic
- Reaproveita personas/estilo de preceptor.server.ts, mas persiste por tema, não só sessão

UI

- Rota /mapa (ou onboarding clínico substituindo o Valley genérico)
- Fluxo: 10–15 min, 1 tema por vez, conduta + confiança
- Visual: mapa com bandas; CTA “Começar Caso do Dia”

Critérios de pronto

- Entrevista grava ≥8 tópicos no banco
- Refresh da página mantém o mapa
- Admin/aluno isolados
- Teste: médico “já sabe X” → X verde sem empurrar teoria

Onda 2 — Preceptor + Caso do Dia 2.0

Nome: Preceptor

O que é: ensina o buraco; manda o Caso do Dia certo.

Como será feito:

Mudança de regra (crítica)

Hoje: mesmo caso para todo mundo (hash do dia). Alvo: fila pessoal a partir de temas vermelhos/amarelos + revisão espaçada.

Schema / lógica

- next_session(user_id) → escolhe case_id ou gera prompt de caso
- Extender review_cards com confidence (sei|acho|nao_sei)
- daily_case_attempts passa a aceitar lesson_id personalizado (não só "o do dia global")
- Manter "Caso da comunidade" opcional (mesmo caso) como ranking social — separado do hábito pessoal

Sessão 2.0 (player)

- 1. Caso primeiro (sem teoria)
- 2. Conduta + confiança
- 3. Se errou / "não sei" → microteoria socrática (Preceptor)
- 4. Reteste gêmeo
- 5. Atualiza mastery_by_topic
- 6. Agenda revisão

UI

- /caso-do-dia lê next_session
- /professores passa a oferecer "continuar do mapa"
- Painel: Índice de Domínio por tema (não só média de sessões)

Critérios de pronto

- Dois alunos com mapas diferentes recebem casos diferentes no mesmo dia
- Acerto + "sei" em tema verde não repete teoria
- Erro atualiza tema para vermelho e agenda reteste
- Portão piloto: ≥70% com 3 sessões na semana 2

Onda 3 — Universidade adaptativa

Nome: Universidade / Protocolos

O que é: profundidade sob demanda; fila do que falta.

Como será feito:

Lógica

- /academy e /academy/$slug ordenam módulos por lacunas do mapa
- Tema verde → "revisar rápido" / pular
- Tema vermelho → aula completa + caso + guia de bolso
- Ao dominar tema → desbloqueia Guia de bolso + oferta ao paciente (elo demanda)

Schema

- Ligar lessons ↔ topics (lesson_topics)
- topic_edges para pré-requisitos

Critérios de pronto

- Dois alunos veem ordem diferente no mesmo curso
- Tema dominado mostra badge + guia
- Protocolos R$ 297 continua sendo o SKU que destrava o cérebro

Onda 4 — Bibliotecário + Estúdio vivo

Nome: Bibliotecário / Estúdio

O que é: se tem material, usa; se não tem, cria com curadoria e grava.

Como será feito:

Loop

```
necessidade (tema vermelho sem lesson)
  → retrieveCorpus(lessons + cases + knowledge_sources)
  → se score alto: reusa
  → se gap: createLessonIfMissing → status=draft
  → fila /editor/curadoria
  → humano assina (CRM) → publish
  → próximo aluno reusa
```

Schema

```
content_gaps
  topic_id, demand_count, status (open|drafting|in_review|published|rejected)

lesson_origins
  lesson_id, source (human|ai_gap|mentor), gap_id?, created_for_user?
```

Reuso de código

- Geração: studioGenerateLesson / lesson-ai.server.ts
- Ligar raiseBarLesson (hoje orphan) no fluxo de gap
- Deduplicação: embedding da necessidade vs. títulos/monografias

Curadoria (regras)

- Rascunho ≠ publicado
- Limiar: N alunos com mesmo gap ou criticidade alta
- Temas sensíveis (off-label, procedimento) → revisão obrigatória
- Métrica: % gaps fechados por reuso vs criação nova

Critérios de pronto

- Gap artificial cria draft automaticamente
- Draft não aparece para outros alunos até publish
- Após publish, segundo aluno com mesmo gap recebe a aula
- Fila de curadoria usável por admin/docente

Onda 5 — Mentores + Matchmaker + take rate

Nome: Mentores / Matchmaker

O que é: comunidade pungente; aulas próprias; mentoria paga; IA indica.

Como será feito:

Produto

- 1. Mentor tem perfil + aulas gravadas (ligadas a courses/lessons com owner_mentor_id)
- 2. Mentoria 1:1 ou turma pequena com preço
- 3. Checkout Stripe (Connect ou cobrança Doutiva + repasse)
- 4. Take rate configurável (ex. 15–20% inicial)
- 5. Matchmaker: matchMentor(user_id) ranqueia por temas vermelhos ∩ specialties, NPS, preço, agenda

Schema

```
mentor_offerings (tipo, preço, duração, vagas)
mentor_bookings (status, amount_cents, take_rate_bps, stripe_*)
mentor_lesson_links (mentor_id, lesson_id)
```

UI

- /mentores público deixa de ser “em breve”
- /mentors autenticado: match “recomendados para você” no topo
- Inbox já existe → estender com pagamento confirmado

Critérios de pronto

- Fluxo request → pagamento → accept → complete
- Take rate aparece no ledger admin
- Matchmaker sugere ≥1 mentor alinhado a tema vermelho
- Mentor sem aula/agenda não sobe no ranking

Onda 6 — WhatsApp + Eventos (proximidade)

Nome: Canal do hábito / Eventos

O que é: Caso do Dia no bolso; comunidade presencial.

Como será feito:

WhatsApp

- Provider (Twilio / Meta Cloud API / Z-API) atrás de adapter
- Opt-in em /settings
- Cron diário (SP timezone) chama next_session e envia resumo + link mágico
- Resposta por botão/lista → webhook → answerDailyCase

Eventos

- Ligar mentores da base ao /evento e catálogo /events
- Pauta do encontro = lacunas agregadas da turma (mastery_by_topic médio)

Critérios de pronto

- Piloto 50: mensagem entregue; ≥50% abrem; ≥30% respondem
- Evento lista mentores reais (não placeholder)

3. Ordem de arquivos novos (referência de engenharia)

```
src/lib/brain/
  brain.server.ts         # orquestrador + tools
  cartographer.server.ts
  preceptor-session.server.ts  # Session 2.0 / next_session
  librarian.server.ts
  studio-gap.server.ts
  matchmaker.server.ts
  types.ts

drizzle/migrations/
  00XX_mastery_by_topic.sql
  00XX_topics_and_edges.sql
  00XX_content_gaps.sql
  00XX_mentor_marketplace.sql
  00XX_whatsapp_dispatch.sql

src/routes/_authenticated/
  mapa.tsx
  editor.curadoria.tsx     # fila gaps
```

Rotas/API existentes a estender (não duplicar): /api/preceptor, /api/tutor, daily-case.functions.ts, studio.functions.ts, mentors.functions.ts.

4. QA por onda (obrigatório)

Onda | Testes mínimos

- 0 · health, login aluno, tutor responde
- 1 · entrevista → mapa persiste; 2 users isolados
- 2 · mapas diferentes → casos diferentes; confiança altera mastery
- 3 · ordem de academy diverge; guia desbloqueia
- 4 · gap → draft → publish → reuso; draft invisível

… +2 linhas no markdown completo

Conta QA aluno sempre; nunca validar só como admin.

5. Métricas (produto)

- Sessões/semana (hábito)
- Retenção D7 / D30
- Delta médio de mastery_by_topic
- NPS
- % gaps resolvidos por reuso
- GMV mentoria + take rate
- Taxa match → booking

Go Protocolos “nova promessa”: D30 > 60%, NPS > 60, ≥70% com 3 sessões/sem2. Go marketplace: ≥1 booking pago real + curadoria de mentores estável.

6. O que explicitamente NÃO fazer no caminho

- Cinco chatbots separados sem memória compartilhada
- Publicar aula IA sem CRM
- Mentor virar Duolingo (hábito diário)

- Take rate sem pagamento na plataforma
- WhatsApp antes do Caso do Dia 2.0 pessoal funcionar no site
- Medir sucesso por “aulas publicadas”

7. Como executamos a partir de agora

- 1. Onda 0 — você cola envs na Vercel (bloqueante humano).
- 2. Em paralelo no código: Onda 1 (schema + Cartógrafo + /mapa).
- 3. Em seguida Onda 2 (next_session + Caso do Dia 2.0 + Session 2.0).
- 4. Ondas 3→6 na sequência dos portões.

Cada onda = branch/PR focado, migração, UI, teste aluno, commit. Documento vivo: atualizar este arquivo ao fechar cada onda.

Fim do plano. Próximo passo de engenharia: Onda 1 (Memória + Cartógrafo), assim que a Onda 0 estiver desbloqueada — ou iniciar schema/UI da Onda 1 já com mocks se a chave ainda não estiver em produção.